

SCHOTTKY DOCUMENTATION

DANNY CALEGARI AND ALDEN WALKER

1. INTRODUCTION

The program `schottky`[2] is a tool for studying the space of two generator iterated function systems generated by complex dilations. It is intended as a companion to the paper[1], which is the theoretical reference to the algorithms implemented in `schottky`. This document describes how to compile and use the program.

2. COMPILATION

On linux, the only thing that should be required is a C++ compiler, the program `make`, and the X11 development headers. If the headers `Xlib.h` and `Xlib.h` are in the subdirectory `X11/` in a location that the C++ compiler searches, everything should be fine. Change lines 5–6 of `ifs_gui.h` if the headers are in a nonstandard location. Similarly, the `X11` library should be in the search path. Adjust the makefile so that it points to the correct place if necessary.

On OSX, the program `XQuartz` (the version of X11 for OSX) needs to be installed. It is available at <http://xquartz.macosforge.org/>. The computer also needs to have XCode tools installed to get the C++ compiler. This is available from the Apple store.

All of the above concerns the interactive program. The headless command-line tools `certify_arc` and `funddom` need no X11 at all, and are built by `make headless`. They are documented in the section “The headless tools” below.

3. LIVE TECHNICAL SUPPORT

If something goes wrong or is confusing, the best thing to do is to email Alden at alden.walker@gmail.com.

4. USAGE

The left panel shows the limit set. The right panel shows parameter space. The check boxes allow the user to change the appearance of the limit set and change what is plotted in parameter space.

4.1. The window itself.

Starting parameter: `schottky` with no argument starts at $s = -0.58 + 0.33i$. A single argument sets the starting parameter instead, in any of the forms the parameter box accepts below, so `schottky 45` starts at 45° on the circle $|s| = 1/\sqrt{2}$ — the twindragon.

Status line: The bottom row of the window reports what the program is doing: progress through a long computation (with a bar), the result of the last operation, and a one-line explanation of whatever control the mouse is

resting on. Hovering is the quickest way to find out what something does; nothing needs to be clicked to read the help.

Escape: aborts a long computation — a parameter-space repaint or an annulus run — without recording the incomplete result.

Quitting: `q` quits, unless a text box has the keyboard focus, in which case it types the letter `q`. The window manager's close button also works.

The parameter box: The box marked `s =` at the bottom left sets the parameter by typing. It accepts `re im`, `re+imi`, a bare angle `deg` in degrees on the circle $|s| = 1/\sqrt{2}$ (the convention the headless tools use), or `deg@r` for an angle on the circle of radius `r`. Unparseable input is reported and leaves the parameter alone. Immediately above it, the parameter is shown to fifteen digits as `re im |s| arg`, which can be selected and pasted into a command line.

4.2. Limit set.

Clicking: Clicking on the panel recenters the drawing.

Depth: Change the depth to which the limit set is drawn. This changes n , where the picture drawn is the image of a point (non-chunky mode) or disk (chunky mode) under all words in f, g of length n .

Auto depth: Dynamically adjust the depth so that the limit set always looks sharp. This is helpful for zooming in a lot. In this mode, the selected depth is ignored.

Zoom: Zoom in or out

Chunky: If chunky is not selected, the drawing is the image of a point in Λ_z under all words of length n . If chunky is selected, the drawing is a union of disks. As explained in [1], if chunky is selected, the drawing is guaranteed to contain the limit set.

uv graph: This mode (1) indicates the labels on all the balls at the given depth (match the drawing depth and the uv graph depth to see this) and (2) connects these vertices as appropriate. The vertices are connected using the following rules: there is an edge connecting a vertex in $f\Lambda_z$ to one in $g\Lambda_z$ exactly when these vertices realize the distance between $f\Lambda_z$ and $g\Lambda_z$ (they are a closest pair). There is an edge connecting two vertices v, w in $f\Lambda_z$ or $g\Lambda_z$ if there is a shortest pair of vertices $a \in f\Lambda_z$, $b \in g\Lambda_z$ such that (1) a prefix of a is a suffix of v and (b) the matching prefix of b is a suffix of w . Paths in this graph essentially construct short hop paths.

Trap: Draw the *trap* that certifies the current parameter, on top of the limit set. A trap is a pair of words u, v — u beginning f , v beginning g — whose balls $u(B)$ and $v(B)$ meet, and whose displacement, renormalized by $s^{-|u|}$, lands strictly inside the trap-like set. Finding one certifies a whole disk of parameters around s inside $\mathcal{M}$, so it is the object that makes a claim about $\mathcal{M}$ true; until now the program could compute one and print its two words, but never show it.

The picture draws $u(B)$ and $v(B)$ as circles, magenta and green, together with enough of their descendants to see the two clusters *interleave*. The interleaving is the content: it is not enough that the two balls overlap, the two Cantor clusters inside them have to link, so that no small change of s can pull them apart. The two are drawn alternately rather than one after

the other, since they overlap and X11 has no transparency, so whichever went second would hide the other exactly where the picture matters.

The spinner is the greatest word length to search. Its default, 16, finds a trap at most parameters; a shorter search is faster but reports nothing where a trap exists only at greater length, which is the usual situation near $\partial\mathcal{M}$. Two failures are reported differently, and the difference is informative: “no trap-like balls at this parameter” means the attractor is not connected there, or $|s|$ is too small, so there is nothing to look for; “no trap with words of length $\leq n$ ” means there was, and the search did not reach it. At the coincidence cores — 45° is one — there is no finite trap at any length, and the second message is the correct answer rather than a failure.

to trap: Take the view to the trap. This is not merely a convenience: the certifying balls have radius about $r|s|^{|u|}$, which for $|u| = 9$ at $|s| = 0.7$ is a few hundredths against a default view three units across — a mark a few pixels wide. The status line says when the trap is too small to see, and this button then frames it.

nIFS: This stands for n -generator IFS. Although right now it is set to simply plot the 3-generator IFS which computes the differences between points in Λ_z . Consider navigating to a renormalization point, turning off chunky and colors, and zooming in to observe the similarity with the set $\mathcal{M}$ described in [1].

2d IFS: This ignores the imaginary part and only works when the real part of the parameter is between 0.6 and 0.68. It plots the affine limit set as described in [1].

4.3. Mandelbrot.

Clicking: Clicking on the panel changes the selected parameter. Clicking and dragging works. Right clicking selects a new parameter and zooms in.

Recenter: Recenter the window on the selected parameter.

Zoom: Zoom in and out

Mesh size: How many true pixels make up one “pixel” in the grid. Setting mesh size smaller makes the picture sharper.

Connected: Plot set $\mathcal{M}$ (run the disconnectedness algorithm). The shade of blue indicates how hard the program had to work to find a pair of disks which intersect at the required depth.

Contains 1/2: Plot set $\mathcal{M}_0$.

Traps: Find traps in a region. This will not work at all until the panel window is quite zoomed in. It functions by far the best at the boundary of $\mathcal{M}$. If there are no trap-like balls to be had in the current window it now says so, rather than appearing to do nothing.

Scale + axes: Draw a scale bar in the corner and the coordinate axes where they fall in view. On by default, because parameter space is self-similar under the renormalization at a landmark: a deep zoom looks very much like a shallow one, and without a scale there is nothing on screen to say which you are looking at. The bar is given a round length — 1, 2 or 5 times a power of ten, whichever makes it about a quarter of the pane — and labelled with that exact length, so the number is always a clean one.

$|s| = 1/2$, $|s| = 1/\sqrt{2}$: Draw either or both of these circles. They bracket the annulus where $\partial\mathcal{M}$ lives: below $|s| = 1/2$ the attractor is a Cantor set, and

at or above $|s| = 1/\sqrt{2}$ it is robustly connected. Several of the controls can therefore only do anything in the band between them, and the circles make that visible. The picture is also cut off at $|s| = 1$, since for $|s| \geq 1$ the maps are not contractions and there is no attractor to ask about.

Colors: Dark shades mean the parameter is in $\mathcal{M}$; white means it is not. With more than one layer switched on the shade shown is the first that applies, in the order given by the small legend beside the checkboxes.

Landmarks: Mark the *renormalization points* — the parameters σ at which there are words u, v of length a and s, t of length b with $\pi(us^\infty, \sigma) = \pi(vt^\infty, \sigma)$, so that the limit-trap mechanism of the paper applies. The spinner sets the complexity bound $a + b \leq N$; the default 7 gives 2421 points and takes about a tenth of a second, while 9 gives 46201 and reads as noise. Only the upper half plane is populated, since conjugate parameters give mirror-image pictures.

Clicking within a few pixels of a landmark *snaps the parameter to it exactly* and reports its reusable specification, for example `landmark lm:--:0 sigma = 0.5 0.5 a=2 b=1 deg=2`. The snap matters: a limit trap exists only at the exact renormalization point, so a hand-placed click would never land on one. That same `lm:` string is what `funddom` takes as a core, so a point found by eye here can be handed straight to a command line. Clicking an already-selected landmark a second time zooms in on it, which is how the asymptotic self-similarity becomes visible.

Limit traps in annulus: With a landmark selected, certify limit traps over one annular fundamental domain of the elliptic curve $E_\sigma = \mathbb{C}^*/\langle \sigma^b \rangle$ and write the figure as `schottky_annulus_NNN.png`, `.eps`, `.pdf` and a `.txt` sidecar. Each point C is colored by the tail length c at which it is first certified, on the viridis ramp: dark for shallow, bright yellow for deep. That ramp is monotone in lightness, so a longer tail always reads as brighter rather than merely as a different hue, and the figure survives being printed in gray. `render_funddom.py` uses the same ramp, from the same table, so a figure made in the interactive program and one made from a `funddom` raster are directly comparable; the two red circles, at radii ρ and $\rho|\sigma|^b$, bound the fundamental domain. This is the asymptotic picture, drawn deep enough in that it is invariant under $C \mapsto \sigma^b C$; both ρ and the depth are derived from the geometry at σ rather than chosen. The percentage of the fundamental annulus covered stays on screen after the run, and everything needed to reproduce the figure goes in the sidecar.

Write window coords: This outputs the window to the terminal.

Write picture: This recomputes the current picture at the given resolution and writes it as `schottky_mand_NNN.png`, `.eps` and `.pdf`, together with a `.txt` sidecar recording the window, the parameter and every enabled layer with its depth, so that the figure can be regenerated later. The counter `NNN` increases on each use, so nothing is overwritten. In the two vector formats the raster is embedded as an image while the overlays — the circle $|s| = 1/\sqrt{2}$ and the current parameter — are emitted as true paths, so they stay sharp at any magnification. The status line at the bottom of the window reports what was written.

Draw path: Allows the user to draw a path by clicking in parameter space.

Find boundary path: Draws a path obtained by following the boundary of the component of Schottky space in which the current parameter lies. This only functions if the selected parameter is *not* in set $\mathcal{M}$.

Path options:

Delete: Delete the path

Find traps along path: Find a sequence of balls of traps to certify that the path lies in the interior of $\mathcal{M}$. This uses the trap depth as selected above.

Create movie along path: Create a movie. This requires that the program `ffmpeg` be installed in the directory `../ffmpeg/`.

Find uv words along path: Finds a sequence of closest pairs of balls along the path.

Find half words along path: This finds balls of points guaranteed to be in set $\mathcal{M}_1$, as described in [1]. This is somewhat tricky to use. It uses the limit set depth to set the initial size of the chunks it finds. Thus, to certify a hole, one should navigate to the hole, zoom in on the limit set and increase the depth until the hole is evident there, then decrease the depth a little. Then draw the path and find half words along it.

5. THE HEADLESS TOOLS

Everything above concerns the interactive program. Two further programs do the same mathematics with no graphics at all:

certify_arc: traps and trap-like balls at a single parameter or along an arc of the circle $|s| = 1/\sqrt{2}$, including the interval-arithmetic certificates.

funddom: limit traps, and how much of a fundamental domain of the elliptic curve $E_\sigma = \mathbb{C}^*/\langle \sigma^b \rangle$ they cover at a renormalization point.

They exist for three reasons. They run where there is no display — over plain `ssh`, in a batch queue, on a machine with no X11 at all. They are scriptable, so a computation can be recorded as a command line and repeated exactly. And they are where the quantitative statements in [1] actually come from: the interactive program is for looking, these are for measuring.

5.1. Building them.

`make headless`

builds both and needs nothing but a C compiler, a C++ compiler and `libm` — no X11 headers, no X11 library, and no other dependency. (`make` on its own builds these two *and* the interactive program, which does need X11.) If you extend the code, note the maintenance rule: anything that draws must go inside `#ifndef IFS_NO_GRAPHICS`, which is the macro that selects the graphics-free objects.

5.2. Conventions.

Angles: Except where noted, a parameter is given as an angle in *degrees* on the circle $|s| = 1/\sqrt{2}$, so `45` means $s = (1 + i)/2$, the twindragon. That circle is where the interesting part of $\partial\mathcal{M}$ lies, which is why it is the default coordinate. Two commands take the parameter directly instead and so reach points off the circle: `dumpltbmanyz`, and `funddom`'s core selectors.

Words: A word in f and g is written outermost letter first. Internally a word is a *bit string*, 0 for f and 1 for g , and that is the form every dump and every certificate log prints, so output can be fed straight back in. The letters **f** and **g** are accepted as well and mean the same thing. Anything else is rejected with a message; it used to be read as a word of all g 's, which produced a confident wrong answer.

Exit codes: 0 means the thing being looked for was found (a trap, a certificate), 1 means it was not, and 2 means the command line was wrong. So these compose in shell scripts.

The two normalizations: This is the one place where it is easy to go wrong. The paper uses $f(z) = sz - 1$, $g(z) = sz + 1$, with base point 0. The program uses $f(x) = zx$, $g(x) = z(x - 1) + 1$, with base point $1/2$. The two frames are conjugate by $x \mapsto (2x - 1)/(1 - s)$, so a trap-like *vector* in one frame differs from the other by a factor of $2/(1 - s)$ — of modulus 2.83 at $s = (1 + i)/2$, and exactly 2 at $s = (1 + i\sqrt{7})/4$. Ball files are written in the program's frame and **funddom** applies the conjugation itself when it reads one, so no user needs to think about this; but anyone writing a new tool against these files does.

5.3. What is certified, and what is merely computed. This distinction matters more than anything else in this section, because the output looks the same either way: a number, and the word CERTIFIED or TRAP. There are three tiers.

Interval-arithmetic certificates: **rigorcert**, **rigorcertseg**, **rigorthin** and **rigorcover** carry out their arithmetic in the sound, outward-rounded interval types of **rigor.h**, and they verify a statement about a whole *box* of parameters rather than about one floating-point sample. When one of these says CERTIFIED, the conclusion holds for every s in the stated arc, subject only to the correctness of the reduction and of the compiler's arithmetic — not to any question of floating-point margin. These are the ones a proof can rest on.

Floating-point trap searches: **point**, **sweep**, **cover**, **dumptlb**, **dumptlbmany**, **dumptlbmany2**, **findtrap**, **mixfind**, **mixscan**, **mixpair**, **mix** and **ckwfind** evaluate rigorous *criteria* in ordinary double precision, with no outward rounding. The theorem behind the margin they print is sound; the computation of it is not verified. These are the right tools for finding a trap, and the wrong tools for claiming one. The intended workflow is to search with these and then verify the answer with **rigorcert**.

funddom: is entirely floating point. It never includes **rigor.h**. Its coverage percentages are numerical evidence — strong evidence, and the shape of the uncovered set is informative — but they are not certificates.

hulltest belongs to none of these: it is a regression test of the convex-hull routine.

5.4. certify_arc. Run it with no arguments for the argument lists; a command given too few arguments says what it needs. Arguments in $\langle \rangle$ are required and those in $[]$ are optional and show their default.

5.4.1. Traps at one parameter.

point: <deg> <n_depth> <uv_depth> <box_halfwidth> [beam=0]

Builds the trap-like ball set T_0 for a box of half-width `box_halfwidth` about s , then looks for a pair of words (u, v) of length at most `uv_depth` whose renormalized displacement lands inside one of those balls. Such a pair is a *trap*, and it certifies not just s but a disk of radius `eps` about it. `beam = 0` searches exhaustively; a positive value uses a beam search of that width, which is much faster and may miss.

```
$ ./certify_arc point 60 16 18 1e-4
s = 0.353553390593274 + 0.612372435695795 i
(|s|=0.707107, arg=60.0000 deg) beam=0
TLB_ok=1 tlb_size=4 certified=1 m=14 eps=6.360285e-07
box_halfwidth=1.000000e-04 eps>=d? 0
```

Read this as: the ball set was built and has 4 balls; a trap was found with words of length $m = 14$; it certifies a disk of radius 6.36×10^{-7} about s . The last field is the question that matters when covering an arc — whether the certified disk is at least as big as the box the ball set was built for, so that the boxes can be chained. Here it is not.

pointz: <re> <im> <n_depth> <uv_depth> <box_halfwidth> [beam=0]

The same computation with the parameter given directly rather than as an angle, so it reaches the rest of $\mathcal{M}$. The circle $|s| = 1/\sqrt{2}$ is the arc the paper works on, but $\mathcal{M}$ is two-dimensional and most of it is nowhere near that circle; there is nothing in the trap machinery that needs s to lie on it.

```
$ ./certify_arc pointz 0.6 0.3 18 16 1e-9
s = 0.6000000000000000 + 0.3000000000000000 i
(|s|=0.670820, arg=26.5651 deg) beam=0
TLB_ok=1 tlb_size=10 certified=1 m=9 eps=9.802885e-05
box_halfwidth=1.000000e-09 eps>=d? 1
```

Note `eps>=d? 1` here, unlike the on-circle example above: the certified disk is larger than the box the ball set was built for, which is the condition that lets a chain of such disks be joined into a covering.

Given the same s to full precision, **pointz** and **point** agree exactly. They are worth comparing once, because `eps` is delicate: feeding **pointz** the fifteen digits that **point** prints, rather than all seventeen, leaves m unchanged but moves `eps` by a factor of two and a half. The certified radius comes from a near-cancellation, so quote s to full precision when it matters.

sweep: <deg_lo> <deg_hi> <steps> <n_depth> <uv_depth>
<box_halfwidth> [beam=0]

The same computation at `steps+1` equally spaced angles, one line each, with a header naming the columns. Useful for seeing where on the circle traps are easy and where they are not.

```
$ ./certify_arc sweep 44 46 4 16 18 1e-4
# sweep |s|=1/sqrt2 n_depth=16 uv_depth=18 box_halfwidth=1.000e-04 beam=0
# arg_deg tlbOK cert m eps min(eps,d) tlbN
44.000 1 1 15 9.3708e-07 9.3708e-07 4
44.500 1 0 -1 0.0000e+00 0.0000e+00 4
45.000 1 0 -1 0.0000e+00 0.0000e+00 8
45.500 1 1 11 4.0141e-05 4.0141e-05 8
46.000 1 1 8 3.7020e-05 3.7020e-05 8
```

The failure at 45° is not noise: the twindragon is one of the two *coincidence cores*, where the trap degenerates and no finite (u, v) of this depth works. Those points need **rigorthin** instead, or the limit traps of **funddom**.

mix: <deg> <n_depth> <uv_depth> <box_halfwidth> [kmax=0]

A diagnostic that runs the ordinary same-length search and the mixed-length variant on the same ball set and compares them. Note that the mixed-length routine is documented in the source as *not sound* for **kmax** ≥ 1 , because it omits a protrusion check; treat its answers as exploration, not certification.

5.4.2. Covering an arc.

cover: <deg_lo> <deg_hi> <n_depth> <uv_depth> <d0> [beam=3000]
[min_gap_deg=5e-4] [prefix=cover]

Walks along the arc, finding a certified disk at each position and advancing by the arc it subtends, so that the union of the disks covers as much of the arc as it can. Reports the fraction covered and lists the gaps, and writes <prefix>_cert.log (one line per disk, with the words that certify it) and <prefix>_gaps.log. It backs off past points where it gets stuck rather than hanging. Floating point throughout, so its output is a *plan* for a rigorous covering, not the covering itself.

beam must be positive here. Unlike **point** and **sweep**, this command always uses the beam search, and a beam of 0 does not mean “exhaustive” — it means no pruning at all, so the frontier grows without bound and the run never finishes. It is refused.

rigorcover: <chunk_id> <deg_lo> <deg_hi> <n_depth> [dmax=0.005]
[dmin=1e-6] [ndmax=n_depth+6] [findnd=12]

The rigorous counterpart, and the heaviest command here: it searches for a trap pair with the floating-point machinery, then *verifies* that pair over a box in interval arithmetic, shrinking the box or deepening the certificate until it succeeds, and grows a certified interval. It checkpoints every certified box to cov_<chunk_id>.jsonl as it goes, so a long run can be killed and resumed, and different chunks can run in parallel in different directories. Expect minutes to hours per chunk.

5.4.3. Verifying a trap you already have.

These take the words as arguments and prove something about a box of parameters. This is the half of the work that a proof cites.

rigorcert: <deg_lo> <deg_hi> <u> <v> <n_depth>

The all-arcs criterion: verifies, in interval arithmetic and for every s in the arc, that around the outer boundary of the union of the two word-disk clouds the labels alternate u, v, u, v , which is what makes (u, v) a trap.

```
$ ./certify_arc rigorcert 83.625 83.626 010010001001 111100110111 6
CERTIFIED min_margin=2.950105e-04 arc=[83.62500,83.62600] (certified)
```

min_margin is how much room to spare the worst of the checks had; it must be positive. Three outcomes are possible: **CERTIFIED**, **not-a-trap** (the criterion fails even in floating point — the pair is wrong), and **interval-FAILED** (it is a trap numerically but the box is too wide for the intervals to separate; halve the arc and try again).

rigorcertseg: <deg_lo> <deg_hi> <u> <v> <n_depth>

A relaxed version of the same idea, in which only four alternating directions are checked rather than every arc. It tolerates individual balls entering and leaving the outer boundary as s moves across the box, which the strict test does not, so it often certifies a wider arc:

```
$ ./certify_arc rigorcertseg 83.625 83.626 010010001001 111100110111 6
CERTIFIED min_margin=7.582363e-04 arc=[83.62500,83.62600] (certified)
```

rigorthin: <deg_lo> <deg_hi> <wA> <wB> <wC> <wD> [theta0_deg=10]

The thin-trap certificate, for the coincidence cores where the ordinary trap degenerates. It verifies that the chord from $wA(0)$ to $wB(0)$ and the chord from $wC(0)$ to $wD(0)$ cross *transversely*, with acute angle at least `theta0`, for every s in the box. That is the finite, machine-checkable half of the argument that the core lies in the interior of $\mathcal{M}$; the contraction lemma that turns it into an exact infinite coincidence is proved by hand.

```
$ ./certify_arc rigorthin 44.9 45.1 01 10 00 11
CERTIFIED min_margin=3.968762e-01 arc=[44.90000,45.10000] theta0=10.0
(transverse-crossing-over-box)
```

Note the margin: 0.4, against 3×10^{-4} for `rigorcert` on a box two hundred times narrower. Transversality of two chords is a far more robust thing to verify than alternation of a whole disk cloud, which is exactly why it works at the cores.

Setting the environment variable `RIGOR_DIAG` makes all three print their intermediate quantities to standard error, which is the way to find out *why* a box failed.

5.4.4. Trap-like ball sets. The trap-like set T_0 is the target that a trap has to hit, and building a good one is usually what decides whether a coverage run closes up. `funddom` reads these files.

dumptlb: <deg> <n_depth> <box_halfwidth>

The analytic construction, at most ten balls: one per gap of the convex hull, five gaps, a few trials each.

```
$ ./certify_arc dumptlb 45 22 1e-9
# analytic trap-like balls (T_0) at arg=45.0000 ok=1
# C=11.656854 Z=0.707107 count=10
# s = 0.50000000000000011 0.5
# center_re center_im radius
0.318538161 -0.191715745 0.060743600
-0.191715745 0.318538161 0.060743600
... eight more ...
```

Ten balls of radius 0.06 cover almost nothing, so this is for inspection, not for coverage.

dumptlbmany: <deg> <n_depth> <box_halfwidth> <ngaps> <ntrials>
<nradial>

The same construction swept much more widely — `ngaps` of the largest hull gaps, `ntrials` sample points across each, `nradial` depths inward — emitting every admissible ball it finds rather than one per gap. At 45° with the settings below it produces 1802 balls of radius up to 0.14, and that is the difference between a fundamental domain closing up and stalling in the nineties. Follow it with `prune_tlb.py`.

dumptlbmanyz: <re> <im> <n_depth> <box_halfwidth> <ngaps>
<ntrials> <nradial>

Identical, but takes the parameter directly, so it reaches renormalization points off the circle — which is how one gets ball sets at the CKW hexahole.

Both of the many forms write a `# s = <re> <im>` provenance line, and `funddom` refuses a file whose parameter does not match the core it was asked about. That check exists because trap-like balls are valid only at their own s , and a mismatched file is otherwise indistinguishable from a correct one — it just quietly gives the wrong answer:

```
$ ./funddom s1 log 24 200 44 0.08 0 x.bin 1 tlb45_pruned.txt
funddom: tlb45_pruned.txt holds trap-like balls for
s = 0.500000000000+0.500000000000i, but this core has
sigma = 0.250000000000+0.661437827766i (differing by 0.298).
Those balls are not valid here and the coverage would be
meaningless.
Regenerate them at sigma, e.g. 'certify_arc dump_tlbmany
0.250000000000 0.661437827766 20 1e-9 40 14 8'.
```

5.4.5. *Searching for trap words.* Given a parameter, these look for the pair (u, v) that `rigorcert` will then verify. They differ in which geometric criterion they use to decide that a candidate pair really is a trap.

```
mixfind: <deg> <max_uv> <n_depth> <kmax> [ratio_goal=1.5]
[min_uv=4] [max_pixels=1024] [dmult=1.2] [verbose=0]
```

The workhorse. Advances the two words independently, allowing their lengths to differ by up to `kmax`, and tests each candidate with a purely topological interleaving criterion on a rasterisation of the two ball clusters. Prints `deg`, whether a trap was found, u , v , their lengths, and the smaller of the two cut-component counts.

```
$ ./certify_arc mixfind 83.6 18 16 1
83.600000 TRAP 010111001110001101 111100111001110110 18 18 119
```

```
mixscan: <max_uv> <n_depth> <kmax> [...]
```

The same, over a list of angles read from standard input — the way to survey an arc.

```
$ printf "45\n60\n83.6\n" | ./certify_arc mixscan 18 16 1
45.000000 none - - 0 0 -1
60.000000 TRAP 0100111000111 1011111111000 13 13 95
83.600000 TRAP 010111001110001101 111100111001110110 18 18 119
```

Again 45° finds nothing, for the reason given above.

```
ckwfind: <deg> <max_uv> <n_depth> <kmax> [...]
```

Finds a topological trap as `mixfind` does, then additionally tests the round-disk criterion of [1], Definition 7.1.3, and reports its margin. The round-disk test is stricter, and a pair can pass the topological test and fail it:

```
$ ./certify_arc ckwfind 83.6 18 16 1
83.600000 TRAP 010111001110001101 111100111001110110 ckwFAIL -0.00163015
```

A negative margin here means the enclosing disks are not separated enough, and that `rigorcert` on this pair will report `not-a-trap`. It is worth running before committing a pair to a long interval-arithmetic verification.

```
mixpair: <deg> <u_bits> <v_bits> <n_depth> [max_pixels=512]
[dmult=6] [verbose=1]
```

Tests one *explicit* pair, printing the ball geometry as it goes. This isolates the geometric test from the search, which is what you want when a pair that ought to work does not.

findtrap: <deg> <max-uv> <max-n> [max-pixels=512] [verbose=1]

The original same-length search, kept as a check on the newer machinery. On the circle $|s| = 1/\sqrt{2}$ it short-circuits, since there the attractor is robustly connected and the question is trivial — so a TRAP from this command at exactly $|s| = 1/\sqrt{2}$ is not saying much.

5.4.6. Self-test.

hulltest: Runs the convex-hull routine on the inputs that used to break it — all-collinear points, duplicates, a single point, and a real depth-6 ball cover — and checks that the result is counter-clockwise and contains every input point. Run it after touching anything in that area; it should print ALL PASS.

5.5. funddom.

funddom <core> <mode> <cmax> <RESX> <RESY> <rho> <survdepth> \
<out.bin> [nper] [balls.txt]

At a renormalization point σ — one where there are words u, v of length a and s, t of length b with $\pi(us^\infty, \sigma) = \pi(vt^\infty, \sigma)$ — the paper's Lemma 9.2.5 gives, for each C , a criterion for the parameters $\sigma + C\sigma^{bn}$ to be trapped for all large n . The criterion is invariant under $C \mapsto \sigma^b C$, so it really lives on the elliptic curve $E_\sigma = \mathbb{C}^*/\langle \sigma^b \rangle$, and covering a single fundamental domain establishes a whole punctured neighborhood of σ in $\mathcal{M}$. This program computes, for each C on a raster, the least tail length c at which C is certified, and reports what fraction of a fundamental domain is covered.

5.5.1. Choosing the core.

s0: $\sigma = (1+i)/2$, the twindragon, $a = 2$, $b = 1$.

s1: $\sigma = (1+i\sqrt{7})/4$, the tame twindragon, $a = 1$, $b = 4$.

hex: the CKW hexahole, $\sigma \approx 0.37186 + 0.51941i$, $a = 8$, $b = 1$. This one lies on $\partial\mathcal{M}$, so it is the control: it must *not* reach full coverage, and if a change to the program makes it do so, the change is wrong.

lm:<A>:[:index]: any landmark point at all, specified by the sign strings of A and B over $\{-, 0, +\}$ (meaning coefficients $-1, 0, +1$) in $Q(z) = A(z)(1 - z^b) + z^a B(z)$, with A beginning $-$; then $a = |A|$ and $b = |B|$. This is exactly the **spec** column that **funddom landmarks** prints, and the string the interactive program reports when you click a landmark, so a point found either way can be handed straight here. Unlike **coin:** it reaches the infinite coincidences: the three named cores are **lm:-+:-:0**, **lm:-:+++:0** and **lm:-+---+++: -:1**.

coin:<u>:<v>[:k]: a *finite* coincidence $u(0) = v(0)$, given as two equal-length f/g words beginning with different letters. The program solves for σ itself and derives everything else.

Run a **lm:** or **coin:** selector with no further arguments and it just reports what it found, which is the quickest way to check a specification:

```
$ ./funddom lm:-+:-:0
lm: A=-+ B=- a=2 b=1 Q(z) = -1 z^0 +2 z^1 -2 z^2
admissible roots (1/2<|z|<1, Im z>0): 1
```

```

[0] 0.5000000000000000+0.5000000000000000i  |z|=0.707106781
      arg=45.00000 deg
sigma = 0.5000000000000000+0.5000000000000000i
|sigma| = 0.707106781187  arg = 45.000000 deg
a = 2, b = 1; Delta = -1.000000000+1.000000000i;
P'(sigma) = 4.000000000-4.000000000i

$ ./funddom coin:fgff:gfgg
coincidence u = fgff , v = gfgg (m = 4)
d = (-1,+1,-1,-1)
polynomial (degree 3): -1 + z - z^2 - z^3 = 0
admissible roots (1/2 < |sigma| < 1, Im sigma > 0): 1
[0] sigma = 0.419643377607081+0.606290729207199i
      |sigma| = 0.737352705760
      arg = 55.311003 deg
      a = 4, b = 1, s = t = f; Delta = 0;
      P'(sigma) = 1.470353793-5.478273590i

```

Only roots with $1/2 < |\sigma| < 1$ and $\text{Im } \sigma > 0$ are admissible: outside that annulus the attractor is either a Cantor set or has no interior, and conjugate parameters give mirror-image pictures. If a polynomial has several admissible roots, `k` or `index` chooses one, counting from zero.

5.5.2. The two modes.

- ann:** the pixels are the C in the square $[-\rho, \rho]^2$, so the picture is an annular neighborhood of σ in the C plane, with the fundamental domain between the circles $|C| = \rho$ and $|C| = \rho|\sigma|^b$.
- log:** the pixels are $x = \arg C$ and $y = \log |C|$, descending from $\log \rho$ through `nper` fundamental periods, each of height $b \log(1/|\sigma|)$. This unrolls the annulus into a ribbon, so one period fills the raster and the resolution is not wasted. It is the right mode for measuring coverage; **ann** is the right mode for a picture.

5.5.3. The other arguments.

- cmax:** the largest tail length c to try, at most 253. Cost grows like $(3/2)^{\text{cmax}}$. There is a real trade-off with ρ : certification at tail length c implies certification of $\sigma^b C$ at $c + b$, so every factor of $|\sigma|^b$ further in costs b more levels, and a small ρ therefore *requires* a large **cmax**.
- RESX RESY:** the raster size.
- rho:** in **ann**, the half-width of the square; in **log**, the outer radius. It must satisfy $\rho \leq (R_\Gamma - |Y_0|)/|dY/dC|$, which the program prints; beyond that bound every sampled C is out of range and the result is vacuous rather than negative. It says so if that happens.
- survdepth:** the depth of the test for whether C lies in T_σ at all, which is what the second percentage reports. Zero does not switch the test off: it leaves the single check $|Y(C)| \leq R_\Gamma$, which is sound but so weak that the column tends to read 100%. At **s0** with $\rho = 0.16$ it gives 100.0000% at **survdepth** 0 against 95.4972% at 8. Use 8 or more when that number is meant to mean something.
- nper:** how many fundamental periods to stack in **log** mode, at most 64. Each is reported separately, which is the direct way to see whether the picture is really scale-invariant.

balls.txt: the trap-like ball set. Optional only for **s0** and **s1**, which have ten balls compiled in — far too few to cover anything, so in practice always supply one.

5.5.4. Enumerating the landmark points.

funddom landmarks <Nmax>

lists every renormalization point of complexity $a + b \leq \text{Nmax}$: the roots in $1/2 < |\sigma| < 1$, $\text{Im } \sigma > 0$ of $Q(z) = A(z)(1 - z^b) + z^a B(z)$ over all A, B with coefficients in $\{0, \pm 1\}$ and $A_0 = -1$. Each line carries everything a coverage run needs, plus the reusable **lm:** specification.

```
$ ./funddom landmarks 4
# sigma_re sigma_im |sigma| arg_deg a b deg
# Delta_re Delta_im Pp_re Pp_im spec
0.5000000000000000 0.5000000000000000 0.707106781 45.00000 2 1 2
+1.0000000000 +4.0000000000 -4.0000000000 lm:--:-0 <= s0 (twindragon)
-0.419643377607081 0.606290729207199 0.737352706 124.68900 1 3 3
... lm:--:-0:0
... sixteen more ...
18 landmark points with a+b <= 4
```

The counts are 1, 18, 99, 533, 2421, 10958, 46201 for $\text{Nmax} = 3, \dots, 9$. Cost grows like $(N - 1)3^{N-1}$ polynomials, so 8 takes under a second, 9 a few seconds and 10 about sixteen. Note that the hexahole first appears at $a + b = 9$.

5.6. File formats.

Ball files: plain text, one ball per line as **center_re center_im radius**, with **#** comments. The line **# s = <re> <im>** records the parameter the balls were computed at and is checked, as described above. Coordinates are in the program's own normalization; **funddom** applies the $2/(1 - \sigma)$ conjugation as it reads. Files of more than 200000 balls are truncated, with a warning saying so and that the resulting coverage is therefore a lower bound.

out.bin: the raster **funddom** writes: no header, two bytes per pixel, in row-major order. The first byte is the least tail length c that certified the pixel, with 255 meaning never certified up to **cmax** and 254 meaning the pixel lies outside the plotted region (which happens only in **ann** mode). The second byte is 1 if C appears to lie in T_σ . Turn it into a picture with **render_funddom.py**.

5.7. The Python helpers.

Each prints its own documentation when run with no arguments.

prune_tlb.py: reduces a ball file to its maximal balls — the ones not contained in another. This is not cosmetic: **dumptlbmany** emits a great many redundant balls, and pruning typically cuts the count by a third to a half while covering exactly the same set, which makes every later coverage run proportionally faster.

render_funddom.py: turns an **out.bin** raster into a PNG, coloring by first certified level: blue for shallow, orange for deep, white for uncertified, gray for outside T_σ . With **--annulus** it draws the two bounding circles of the fundamental domain. **render_funddom.py bar out.png** writes the color key.

holes.py: rigorous detection of holes in $\mathcal{M}$, via $s \in \mathcal{M} \iff 2/s \in \Lambda - \Lambda$.

`spiral.py`: follows the renormalization spiral $\sigma + C\sigma^{bn}$ at a renormalization point.
`diffset_test.py`: asks whether a given difference is an *interior* point of $\Lambda - \Lambda$, which is the difference-set signature of $s \in \text{int } \mathcal{M}$.
`diffset_selfcover.py`: a prototype of the difference-set self-covering argument. It is a prototype and has a known limitation, documented in its own header.

5.8. Use cases.

5.8.1. *Is this parameter in the interior of $\mathcal{M}$?* Search, then verify. Find a trap pair, check its round-disk margin, and only then spend interval arithmetic on it:

```
./certify_arc mixfind 83.6 18 16 1          # -> TRAP with an explicit (u,v)
./certify_arc ckwfind 83.6 18 16 1          # -> round-disk margin positive?
./certify_arc rigorcert 83.625 83.626 010010001001 111100110111 6
```

If the last step says `interval-FAILED` the pair is right and the box is too wide: halve it. If it says `not-a-trap` the pair is wrong: search again, or try `rigorcertseg`, whose criterion is weaker and which certifies wider boxes.

5.8.2. *Certify a whole arc.* Plan the covering in floating point first, then do it rigorously:

```
$ ./certify_arc cover 59.9 60.1 16 18 1e-4
# cover [59.9000,60.1000] deg n=16 uv=18 d0=1.00e-04 beam=3000
# min_gap=5.0e-04 deg
boxes=11 balls=233 checks=247
covered 0.064000 of 0.200000 deg (32.0000%);
  4 gap-interval(s), total 0.136000 deg
  GAP [59.90000, 60.02750] deg (0.12750 deg)
  GAP [60.08102, 60.08152] deg (0.00050 deg)
  GAP [60.08464, 60.08514] deg (0.00050 deg)
  GAP [60.08542, 60.09292] deg (0.00750 deg)
certificate log: cover_cert.log ; gaps: cover_gaps.log
```

Doing this first is worth it because it is far cheaper and it says where the hard points are: here a tenth of a degree at the low end resists entirely, and raising `n_depth` and `uv_depth` is what to try next. `cover_cert.log` records the actual (u, v) for every disk, so each line of it can be handed to `rigorcert` independently. Widening the arc costs time quickly — a few degrees can run for many minutes — so start narrow.

Then `rigorcover` does the real work in interval arithmetic:

```
./certify_arc rigorcover 1 59.9 60.1 16
```

checkpointing every certified box to `cov_1.jsonl`, so it can be killed and resumed. Different chunk ids in different directories run in parallel.

5.8.3. *Does a renormalization point have a punctured neighborhood in $\mathcal{M}$?* This is the limit-trap computation, and the ball set is the thing that matters. At the twindragon:

```
./certify_arc dumptlbmany 45 22 1e-9 40 14 8 > tlb45.txt
python3 prune_tlb.py tlb45.txt tlb45_pruned.txt
./funddom s0 log 24 700 155 0.08 0 s0.bin 1 tlb45_pruned.txt
```

which reports

```

read 1118 trap-like balls from tlb45_pruned.txt
core s0: sigma=0.500000000+0.500000000i a=2 b=1
Delta=-1.000000+1.000000i P'(sigma)=4.000000-4.000000i
Y0 = -2.000000-2.000000i |Y0|=2.828427
dY/dC = 8.000000+8.000000i (|.|=11.313708)
R_Gamma=6.828427 trap-like balls: 1118,
radii 0.3891 0.3891 0.3891 0.3891 0.3526 0.3526
s0 log cmax=24 rho=0.08 nper=1 :
covered 99.9991% in T_sigma 100.0000% (n=108500)
period 0 (|C| ~ 0.05657..0.08000): covered 99.999% in T_sigma 100.000%

```

— a fundamental domain of E_σ essentially covered, so the twindragon has a punctured neighborhood in $\mathcal{M}$. The cumulative histogram printed underneath shows where the work went: half the domain is certified by tail length 3, and the last hundredth of a percent needs twenty.

Now the control. The hexahole is on the boundary, so the same computation must fail:

```

./certify_arc dumptlbmanyz 0.371858680074136 0.519411153747943 \
20 1e-9 40 14 8 > tlbhex.txt
python3 prune_tlb.py tlbhex.txt tlbhex_pruned.txt
./funddom hex log 24 400 90 0.0003 26 hex.bin 1 tlbhex_pruned.txt
# covered 99.7389% in T_sigma 99.8806%

```

and it does: the coverage stalls just under the T_σ ceiling and stays there however far `cmax` is pushed, because the uncovered part is scale-invariant — it is the hole spiral. A coverage figure means nothing without this comparison, since 99.7% looks like success until one knows that the answer for a boundary point is also 99.7%.

5.8.4. Make a figure for a paper.

```

./funddom lm:-+:-:0 ann 27 700 700 0.005 8 fig.bin 1 tlb45_pruned.txt
python3 render_funddom.py fig.bin 700 700 fig.png \
--annulus 0.005 0.707106781 1

```

Use `ann` for a picture and `log` for a number. Choose ρ small enough to be inside the asymptotic regime, where the picture is genuinely invariant under $C \mapsto \sigma^b C$ — about $\rho_{\max}/70$ is a reasonable rule — and remember that a small ρ needs a large `cmax`. The interactive program’s “Limit traps in annulus” button does all of this at a selected landmark and writes PNG, EPS and PDF together with a sidecar recording the settings, which is usually less work.

REFERENCES

- [1] D. Calegari, S. Koch, and A. Walker *Roots, Schottky semigroups, and a proof of Bandt’s conjecture*, preprint: arXiv: 1410.8542
- [2] D. Calegari and A. Walker, *schottky*, computer program available from the authors’ web-pages.

DEPARTMENT OF MATHEMATICS, UNIVERSITY OF CHICAGO, CHICAGO, ILLINOIS, 60637
E-mail address: `dannyc@math.uchicago.edu`

DEPARTMENT OF MATHEMATICS, UNIVERSITY OF CHICAGO, CHICAGO, ILLINOIS, 60637
E-mail address: `alden.walker@gmail.com`